

EdTech Platform

Comprehensive Project Documentation

Generated on: June 12, 2026

Project Information:

Project Name: EdTech Platform

Version: 1.0.0

Type: Full-Stack Web Application

Status: Active Development

Table of Contents

- 1. Project Overview
- 2. Technology Stack
- 3. System Architecture
- 4. User Roles & Access Control
- 5. Features & Functionality
- 6. Application Flow
- 7. Database Models & Schema
- 8. API Endpoints
- 9. Frontend Structure
- 10. Backend Structure
- 11. Security & Best Practices
- 12. Deployment & Setup
- 13. Future Enhancements
- 14. Troubleshooting & Support

1. Project Overview

The EdTech Platform is a comprehensive web-based learning management system designed to provide an adaptive learning experience for students. The platform supports multiple subjects including Backend, Cloud, Frontend, Node.js, and React development. It features role-based access control with distinct interfaces for Students and Administrators, adaptive learning tracks, comprehensive analytics, and interactive video-based content delivery.

Key Objectives:

- Provide adaptive learning paths for different skill levels
- Track student progress and performance metrics
- Enable instructors to manage courses and monitor analytics
- Support interactive content delivery through videos and assessments
- Facilitate communication through query/support system
- Provide data-driven insights through comprehensive analytics

2. Technology Stack

Backend Framework: Express.js (Node.js)

Database (Primary): MongoDB

Database (Secondary): MySQL

Frontend Framework: React 18.2.0

Routing: React Router DOM 6.22.3

UI Components: Lucide React Icons

Build Tool: Vite 5.2.0

Authentication: bcrypt (password hashing)

Testing: Jest 30.3.0

Environment Management: dotenv

3. System Architecture

3.1 Architecture Overview

The EdTech Platform follows a three-tier client-server architecture with clear separation of concerns:

- Frontend Tier: React.js single-page application (SPA) with React Router
- Backend Tier: Express.js REST API server
- Data Tier: MongoDB (primary) and MySQL (secondary) databases

3.2 High-Level Components

Frontend (React): Single-page application handling UI/UX, routing, state management

API Server (Express): RESTful API handling business logic and data processing

Database Layer: MongoDB for analytics and course data, MySQL for user management

Authentication: Session-based with bcrypt password hashing

File Storage: Static files served through Express (videos, assets)

4. User Roles & Access Control

4.1 Role Hierarchy

Admin: Complete system access with management capabilities

Student: Limited access to courses and personal learning path

Unauthenticated: Public pages only (About, Contact, Auth)

4.2 Admin Features

- User Management - View all registered students
- Course Management - Upload and manage courses
- Analytics Dashboard - Monitor performance metrics
- Query Management - View and resolve support queries
- Video Analytics - Track consumption patterns
- Recommendation Engine - View adaptive learning recommendations

4.3 Student Features

- Course Access - Browse and enroll in courses
- Learning Tracks - Choose Beginner or Adaptive paths
- Video Content - Watch with interactive controls
- Assessment - Take quizzes and tests
- Progress Tracking - View learning progress
- Query Submission - Submit support queries
- Dashboard - Personal learning path view

5. Features & Functionality

5.1 Core Features

- Authentication System - Secure login/registration with bcrypt
- Adaptive Learning - Intelligent path recommendation
- Multi-Track Learning - Beginner and Adaptive tracks
- Course Management - Modular course structure
- Video Analytics - Detailed tracking of video consumption
- Performance Analytics - Comprehensive metrics
- Query System - Support ticket system
- Responsive Design - Mobile-friendly interface
- Error Handling - Graceful error management
- Protected Routes - Role-based access control

5.2 Subject Coverage

The platform supports courses in: Backend Development, Cloud Computing, Frontend Development, Node.js, React.js

6. Application Flow

6.1 User Authentication Flow

1. User visits application (unauthenticated)
2. User redirected to Authentication page
3. User chooses Login or Register
4. For Login:
 - Enter credentials
 - System verifies against database
 - Session created and stored in localStorage
 - User redirected to dashboard
5. For Register:
 - Enter user information
 - System creates account
 - User can then login

6.2 Student Learning Flow

1. Student logs in and views Dashboard
2. Sees available courses/subjects
3. Selects course and learning track (Beginner/Adaptive)
4. Enrolls and analytics session created
5. Accesses course content (videos, lessons, quizzes)
6. System tracks performance data
7. Based on performance, system makes recommendations
8. Student views progress on Dashboard
9. Can submit support queries

6.3 Admin Management Flow

1. Admin logs in with admin credentials
2. Redirected to Admin Dashboard
3. Access multiple tabs: Users, Courses, Analytics, Queries, Videos
4. Can perform: view analytics, upload courses, resolve queries
5. All actions logged and stored

7. Database Models & Schema

7.1 MongoDB Collections

Course Collection

Structure: { subjectId, courseTitle, modules[] }

- modules[]: title, lessons[]
- lessons[]: lessonId, title, type, contentHtml, contentUrl, questions[]

Subject Collection

Structure: { subjectId, title, description }

- Stores metadata about available courses

Analytics Collection

Structure: { username, subjectId, date, telemetry[], decision[], track }

- Tracks student performance and learning paths
- telemetry[]: concept, correct, timeSeconds, changes, trigger
- track: "Beginner" or "Adaptive"

VideoAnalytics Collection

Structure: { username, subjectId, lessonId, sessionId, duration, totalSeeks, highSpeedSeconds, status, events[], date }

- Detailed video interaction tracking
- Events: play, pause, seek, completed, abandoned, etc.

7.2 MySQL Tables

Users Table

Columns: id, username, email, password (hashed), contact_no, created_at

- Stores authentication credentials
- Passwords hashed with bcrypt

8. API Endpoints

8.1 Authentication Routes (/api/auth)

POST /register - Register new user

POST /login - Authenticate user

GET /users - Get all users (Admin)

8.2 Course Routes (/api/courses)

GET /subjects - Get all subjects

GET /details/:subjectId - Get course details

POST /upload - Upload course (Admin)

8.3 Analytics Routes (/api/analytics)

POST / - Save analytics session

POST /enroll - Enroll in track

POST /video - Record video telemetry

GET /user/:username - Get user analytics

9. Frontend Structure

9.1 Main Pages & Routes

Home (/) - Landing page for authenticated users

Auth (/auth) - Login and registration

Dashboard (/dashboard) - Learning path management

CoursePlayer (/player/:subjectId) - Video playback

Assessment (/assessment/:subjectId) - Quizzes

Admin (/admin) - Admin dashboard

9.2 Key Components

- **AuthContext** - Global authentication state management
- **ErrorBoundary** - Error handling and recovery
- **Header** - Navigation and user menu
- **Footer** - Footer information
- **Modal** - Reusable dialog component
- **ProtectedRoute** - Role-based route protection

10. Backend Structure

10.1 Directory Structure

backend/

% % % server.js (Entry point)
% % % config/db.js (Database setup)
% % % models/ (Database schemas)
% % % routes/ (API handlers)
% % % scripts/seed.js (Data seeding)
% % % data/ (Course data files)

10.2 Middleware Stack

- CORS - Enable cross-origin requests
- JSON Parser - Parse incoming JSON
- Static File Server - Serve frontend and assets
- Error Handler - Global error handling

10.3 Route Organization

Authentication Routes: User login, registration, user management

Course Routes: Subject listing, course details, course uploads

Analytics Routes: Performance tracking, video analytics, recommendations

Query Routes: Support ticket management

11. Security & Best Practices

11.1 Authentication & Security

- Password Hashing - bcrypt with 10 salt rounds
- Session Management - localStorage for persistence
- Protected Routes - Role-based access control
- CORS Protection - Restrict cross-origin requests
- Environment Variables - Sensitive data in .env
- Error Handling - Server-side logging only

11.2 Best Practices

- RESTful API Design - Standard HTTP methods
- Separation of Concerns - Clear layered architecture
- Environment Configuration - Environment-specific settings
- Error Boundaries - React error recovery
- Logging - Server-side monitoring
- Modularity - Reusable code components

12. Deployment & Setup Guide

12.1 Prerequisites

- Node.js and npm
- MongoDB server or MongoDB Atlas
- MySQL server
- Git version control

12.2 Installation

1. Clone repository:
`git clone <repo-url>`
`cd EdTech-Platform`
2. Install dependencies:
`npm install`
3. Setup environment variables in `.env`:
`MONGODB_URI=<your-mongodb-url>`
`MYSQL_HOST=<host>`
`MYSQL_USER=<user>`
`MYSQL_PASSWORD=<password>`
`PORT=3000`
4. Seed database (optional):
`npm run seed`
5. Run development:
`npm start (backend)`
`npm run dev (frontend)`
6. Build for production:
`npm run build`

12.3 Testing

Run tests with: `npm test`

13. Future Enhancements & Roadmap

1. Advanced ML-based analytics and predictions
2. Real-time WebSocket notifications
3. Adaptive bitrate video streaming
4. Digital certification system
5. Discussion forums and peer learning
6. Native mobile applications
7. Payment integration for monetization
8. OAuth social login integration
9. Full-text search capabilities
10. Analytics report generation and export
11. Internationalization (i18n)
12. Advanced caching and optimization

14. Troubleshooting & Support

14.1 Common Issues

Database Connection Error: Verify .env file has correct DB credentials

Port Already in Use: Change PORT in .env or kill process on port 3000

Module Not Found: Run npm install to install dependencies

CORS Errors: Verify CORS middleware in server.js

Build Fails: Clear node_modules and reinstall: npm install

14.2 Performance Tips

- Enable gzip compression for responses
- Implement response caching
- Optimize database indexes
- Use CDN for static assets
- Implement lazy loading
- Monitor query performance

Thank You!

EdTech Platform - Complete Documentation

Document Generated: 6/12/2026, 12:20:20 PM

For more information and updates, visit the [project repository](#)